基于下一代Ascend平台的纯SIMT编程介绍

作者 张浩、吴林玉

时间 2026/03

课程背景与目标

课程背景:

随着昇腾AI处理器架构的持续演进，下一代架构在硬件设计上增加了SIMT计算单元，用于提升离散操作场景及复杂分支场景的编程易用性;

课程目标:

- 掌握昇腾下一代的SIMT架构;
- 熟悉基于下一代芯片的Ascend C纯SIMT编程模型;
- 熟悉基于下一代芯片的SIMT算子设计、开发的流程;

前置知识:

业界SIMT通用编程方法;

CANN

目录

Part 1 下一代硬件架构回顾

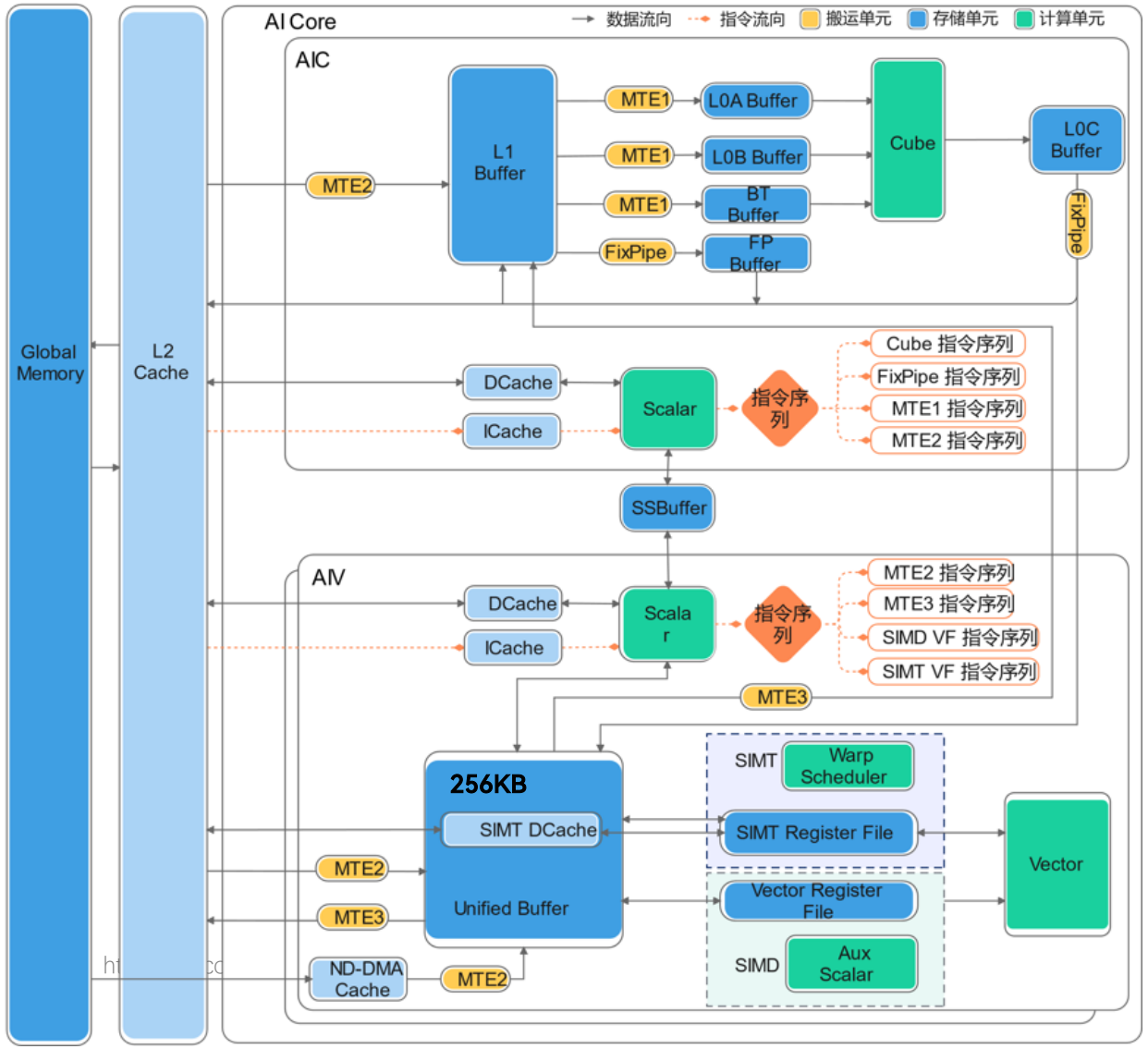
Part 2 SIMT编程模型介绍

Part 3 SIMT算子编程实操

Part 4 SIMT算子优化技巧

下一代硬件架构回顾

相比当代A2/A3，下一代AIV新增SIMT编程能力，可支持纯SIMT编程方式



AIV:

- 可支持两种工作模式：SIMT和SIMD
- **SIMT工作模式：**
  - 支持直接读写GM或UB (UB当做Shared Memory)
  - 每个AIV有4个Warp Scheduler, scheduler id = warp id % 4
  - 每个线程可使用的寄存器受线程个数限制
  - 最大支持128KB Data Cache，Data Cache直接复用UB作为Cacheline

线程个数	寄存器个数
2048	16
1024	32
512	64

CANN

目录

Part 1 下一代硬件架构回顾

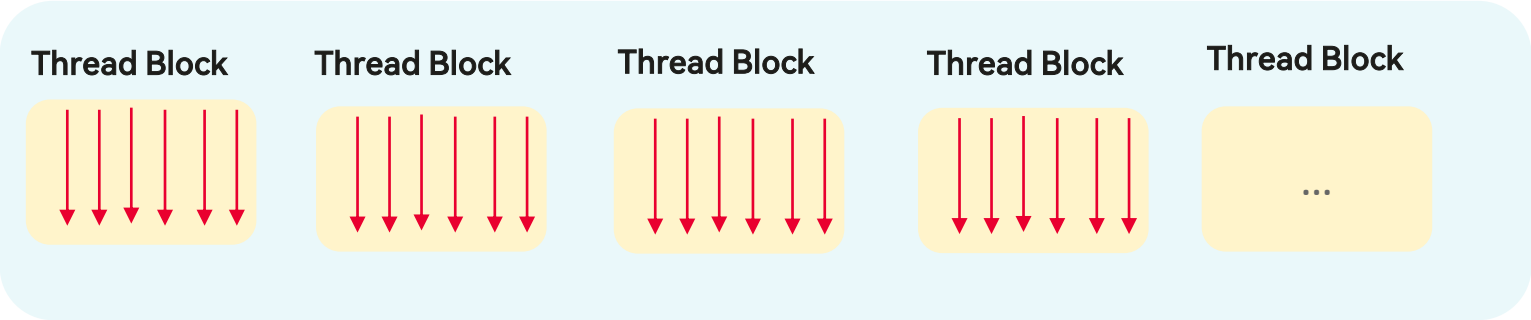
Part 2 SIMT编程模型介绍

Part 3 SIMT算子编程实操

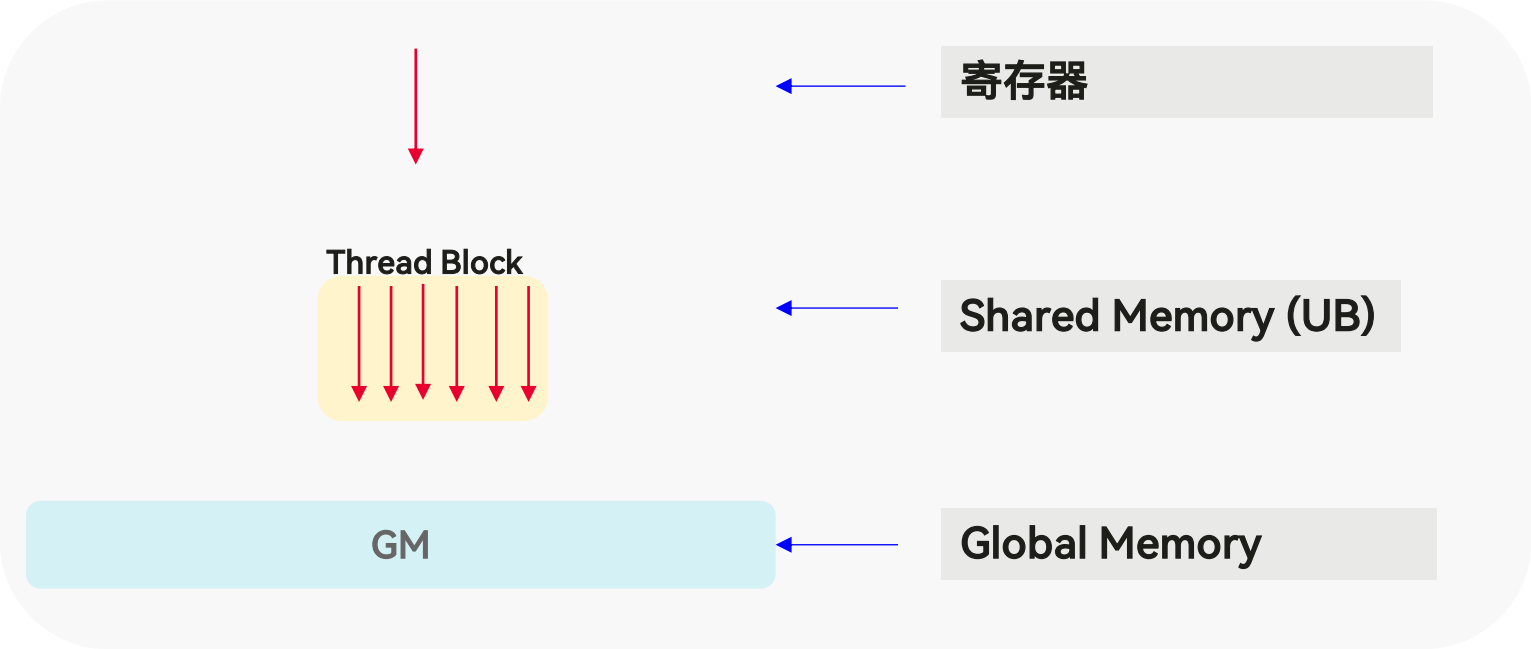
Part 4 SIMT算子优化技巧

Ascend C 纯SIMT编程模型介绍：遵循业界通用的线程架构和三级内存模型

下一代 SIMT 线程架构



下一代 内存模型



核心概念：

- Grid：就是我们一整个核函数的所有线程集合
  - gridDim.x/y/z：grid 有多少 block
  - blockIdx.x/y/z：block 在 grid 内的位置
- ThreadBlock：线程块，对应执行单元AIV
  - blockDim.x/y/z：每个 block 的大小
  - threadIdx.x/y/z：线程在 block 内的位置
- Warp：最小调度和执行单元(固定为32 线程)
- Thread：线程之间只有数据不同，指令完全一致，拥有独立的寄存器和栈

- SIMT支持直接读写GM或UB
- 同一时刻一个AIV只能执行一个thread block
- 一个Thread Block 最多支持**2048**个线程；
- 一个Thread Block中SIMT RegFile总大小128K;

线程个数	寄存器个数
2048	16
1024	32
512	64

CANN

当前SIMT是业界SIMT功能子集，不支持Texture/Surface、cluster、TMA、 Independent Thread Scheduling 、Warp Specialization

Ascend C纯SIMT编程模型介绍：UB内存分配

在SIMT编程中，GM的访问需要使用部分UB空间作为DataCache，因此对于UB地址进行了分段设计。下图详细描述了SIMT编程场景中的内存分布

静态内存，通过数组分配 。
如_ubuf_ half staticBuf[1024];

通过<<<>>>中**dynUBufSize** 指定的**动态内存**大小空间，可通过以下方式申请使用：
使用extern _ubuf_ half dynamicBuf[];

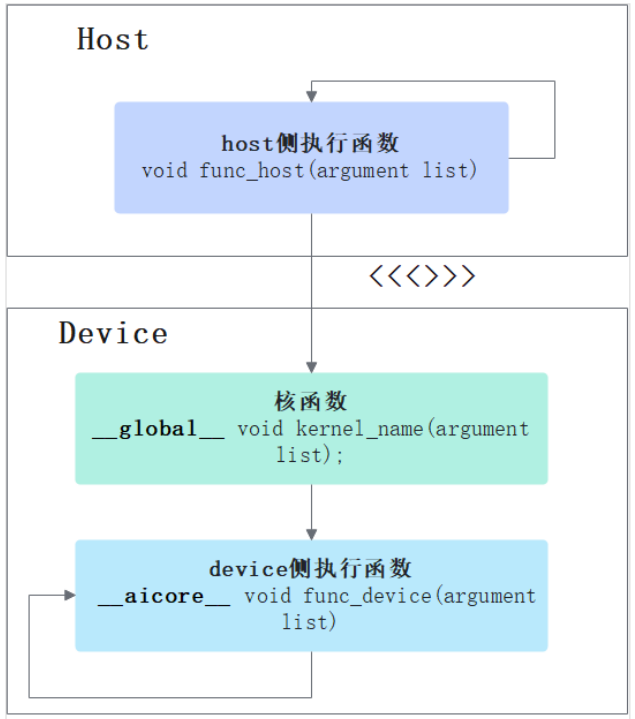
编译器和ASC**预留空间**，大小为8K;

SIMT 用于**Data Cache**的空间，必须大于或等于32K

DataCache = UB总大小(256K) – 静态内存 – 动态内存 – 预留空间(8k)

若DataCache小于32k，则运行时会校验报错

Ascend C纯SIMT编程模型介绍：异构并行计算模型



核函数定义：

__global__ __launch_bounds__(N) void add_custom(float* x, float* y, float* z, uint64_t total_length)

- 使用函数类型限定符__global__，标识它是一个核函数。核函数必须具有void返回类型。
- 函数入参支持的类型如下：基础数据类型，如int32_t、float等。基础数据类型的指针类型，如int32_t*、float*等，实际上这些指针指向的是GlobalMemory内存。
- __launch_bounds__（可选）：设置核函数的最大线程数，线程数越大，每个线程可用寄存器越少，默认值为1024。

核函数调用：

kernel_name<<<block_num, thread_num_per_block, dyn_ubuf_size, stream>>>(args...)

- block_num：为核函数配置的线程块的个数(gridDim)，支持int32_t和dim3类型；
- thread_num_per_block：每个线程块内的线程数量，支持int32_t和dim3类型；
- dyn_ubuf_size：动态申请内存空间总大小；
- stream：用于host和device侧的流同步。

```
__global__ __launch_bounds__(N) void add_custom(float* x, float* y, float* z, uint64_t total_length)
{
    int32_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= total_length) {
        return;
    }
    z[idx] = x[idx] + y[idx];
}
...
add_custom<<<block_num, thread_num_per_block, dyn_ubuf_size, stream>>>(x_device, y_device, z_device, x.size());
```

CANN

完整样例：SIMT实现的Add样例

SIMT编程模型介绍： API接口目录结构

- **SIMT API**
  asc_simt.h
  - **Math 接口**
    simt_api/math_functions.h
    - __aicore__ inline float asinf(float x)
    - __aicore__ inline float atanf(float x)
    - ...
  - **warp类接口**
    device_warp_functions.h
    - Warp Vote
      - int32_t asc_all(int32_t predicate)
      - int32_t asc_any(int32_t predicate)
      - ...
    - Warp Shuffle
      - int64_t asc_shfl_down(int64_t var, uint32_t delta, int32_t width = warpSize)
      - float asc_shfl_up(float var, uint32_t delta, int32_t width = warpSize)
      - ...
    - Warp Reduce
      - int32_t asc_reduce_add(int32_t val)
      - float asc_reduce_max(float val)
      - inline float asc_reduce_min(float val)
      - ...
  - **原子函数**
    device_atomic_functions.h
    - int32_t asc_atomic_add(__ubuf__ int32_t *address, int32_t val)
    - inline uint64_t asc_atomic_xor(__gm__ uint64_t *address, uint64_t val)
    - ...
  - **同步接口**
    device_sync_functions.h
    - inline void asc_syncthreads()
    - inline void asc_threadfence()
    - inline void asc_threadfence_block()
  - **其他**
    - vector 短向量接口
      vector_functions.h
      - short4 make_short4(short x, short y, short z, short w)
      - float2 make_float2
      - ...
    - 其他数据类型接口
      asc_bf16.h、asc_fp8.h、asc_fp16.h
    - 常量、内置宏等

- 线程束（warp）是调度的基本单位，包含32个线程
- warp类接口提供了一些特定的函数和操作，用于在warp内的线程之间进行协作和通信
  - **Warp Shuffle：** 允许warp内的线程之间交换数据
  - **Warp Vote：** 用于在warp内的线程之间进行投票，以确定某些条件是否满足。
  - **Warp Reduce：** 用于在warp内进行归约操作，例如求和、最大值、最小值等。

- 原子操作用于多线程对共享资源的访问场景，避免数据竞争和确保数据一致性。

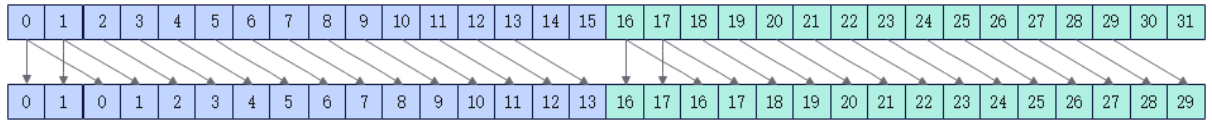
- 线程内的一般操作是相互独立的，某些应用场景中，需要支持线程间的同步
  - asc_syncthreads：等待当前线程块内所有线程代码都执行到该函数位置。
  - asc_threadfance：用于保证不同线程对同一份全局、共享内存的访问过程中，写入操作的时序性，不会阻塞线程，仅保证内存操作的可见性顺序。
  - asc_threadfance_block：用于协调同一个线程块内的线程之间的内存操作顺序。确保在同一线程块内的线程对内存操作的可见性顺序。

SIMT编程模型介绍： Warp类接口

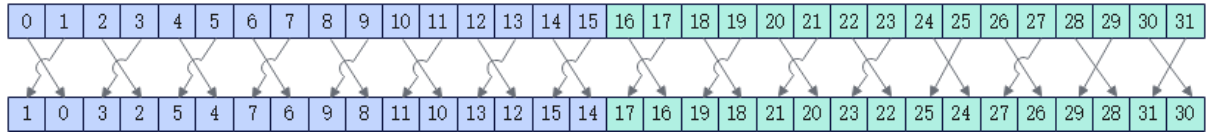
warp类接口提供了一些特定的函数和操作，用于在warp内的线程之间进行协作和通信

- **Warp Shuffle**：允许warp内的线程之间交换数据

asc_shfl_up(LaneId, 2, 16)



asc_shfl_xor(LaneId, 1, 16)



- **Warp Vote**：用于在warp内的线程之间进行投票，以确定某些条件是否满足。

int32_t asc_all(int32_t predicate)

int32_t asc_any(int32_t predicate)

uint32_t asc_ballot(int32_t predicate)

- **Warp Reduce**： 用于在warp内进行归约操作，例如求和、最大值、最小值等。

float asc_reduce_add(float val)

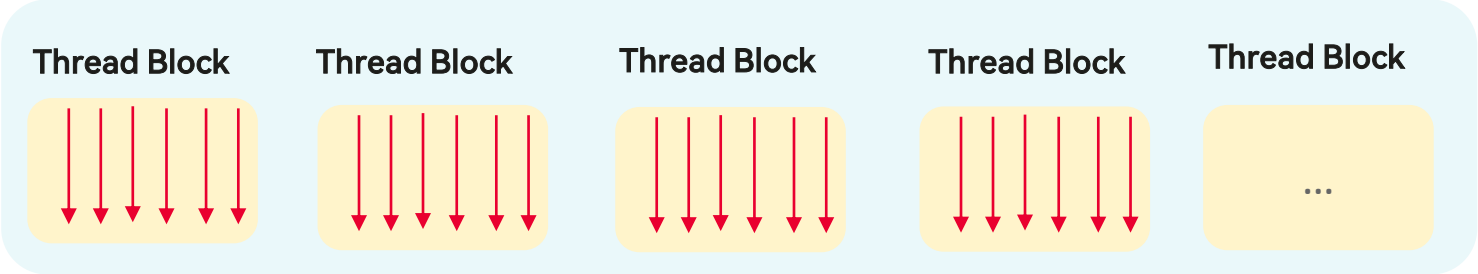
inline half asc_reduce_max(half val)

inline int32_t asc_reduce_min(int32_t val)

CANN

SIMT编程模型介绍： 线程同步及内存屏障接口

同一个 ThreadBlock 内的所有线程运行在同一个AIV核 上，他们之间可通过共享内存通信(UB/GM)，但是线程执行顺序是乱序的



□ 线程同步

使用asc_syncthreads指令保证 ThreadBlock 内所有线程都执行到该指令

```
// 使用静态数组空间，将部分数据缓存到核内
__ubuf__ float input_buf[INPUT_SIZE];

  for (int i = 0; i< bufLen; i++) {
     input_buf[threadIdx.x + THREAD_COUNT * i] = input[idx];
  }
  // ……….

  // ThreadBlock内线程乱序执行，且对input_buf存在数据依赖，因此必须使用同步指令
  asc_syncthreads();
  output[idx] = input_buf[gather_idx];
```

□ 内存屏障

线程在fence前的所有内存读写操作必须完成，且对指定范围的线程可见后，才能执行栅栏后的内存操作

void asc_threadfence()

void asc_threadfence_block()

```
src[0] = src[0] + 1;
// asc_threadfence()保证本线程的写操作顺序对全局可见
asc_threadfence();
dst[0] = src[0];
```

CANN

SIMT编程模型介绍： 语法特性总览

类别	ASC 编程API
内置变量	① gridDim、blockIdx、blockDim、threadIdx、warpSize
内置函数/内存修饰符	① __global__、__aicore__ 、__host__ ② __ubuf__、
Launch Bound	__launch_bounds__(maxThreadsPerBlock)
核函数定义	__global__ __launch_bounds__(maxThreadsPerBlock) void myKernel(float* input, float*output, int size);

核函数调用	dim3 numBlocks; //GridDim;
	dim3 threadsPerBlock; //BlockDim
	numBlocks 和 threadsPerBlock 同时支持int或dim3 类型;
	kernel_name<<<numBlocks, threadsPerBlock, dynUBufSize, stream>>>(args...)
	
	● dynUBufSize : 动态申请内存空间总大小

Memory Fence	__aicore__ inline void asc_threadfence()、asc_threadfence_block()
同步函数	__aicore__ inline void asc_syncthreads()
Warp Vote	int32_t asc_all(int32_t predicate) 、asc_any(int32_t predicate)、 asc_activemask()
Warp Shuffle	__aicore__ inline int32_t asc_shfl(int32_t var, int32_t srcLane, int32_t width = warpSize)、asc_shfl_up、asc_shfl_down、asc_shfl_xor
Warp Reduce	__aicore__ inline unsigned asc_reduce_add(unsigned val) 、asc_reduce_max、asc_reduce_min
原子函数	__aicore__ inline int_t asc_atomic_add(int *address, int val) 、asc_atomic_sub、asc_atomic_exch等_
Math函数	__aicore__ inline float asinf(float x)、__aicore__ inline float atanf(float x) 等

本章小结

- **介绍Ascend C 纯SIMT的编程模型**
  - ✓ SIMT线程架构模型
  - ✓ SIMT UB内存分配
  - ✓ 介绍SIMT的异构计算模型，展示核函数的定义和调用示例
  - ✓ SIMTAPI接口目录结构
  - ✓ 介绍SIMT的核心语法特性
  - ✓ 介绍线程同步、内存屏障、warp等典型接口

CANN

目录

Part 1 下一代硬件架构回顾

Part 2 SIMT编程模型介绍

Part 3 SIMT算子编程实操

Part 4 SIMT算子优化技巧

CANN

SIMT算子编程实操：以gather算子为例

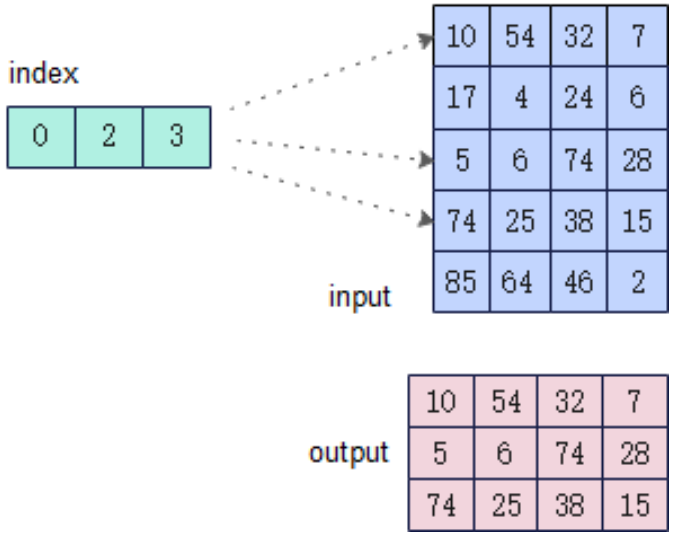
算子功能:

从形状为M * N的二维向量input中获取index指定索引的m行数据

计算公式:

$$output_i = input_{index_i}, \quad index_i \in [0, len(input))$$

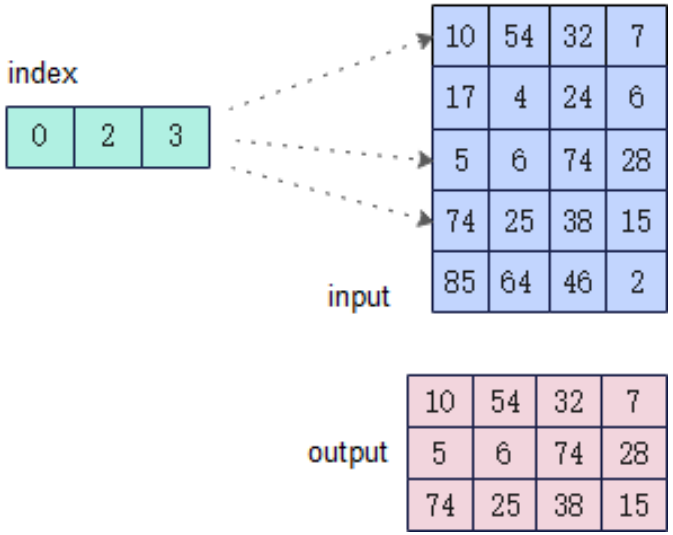
	name	shape	data type
算子输入	input	(M, N)	float
	index	m	uint32_t
算子输出	output	m, N	float



完整样例代码：纯SIMT实现gather算子

CANN

SIMT算子编程实操： 以gather算子为例



核函数实现：

float* input： 输入input的gm指针

uint32_t* index：输入index的gm指针

float* gather_output：输出的gm指针

uint32_t in_width：输入input每行的元素个数

uint32_t index_total_length：输入index的元素个数

```
__global__ __launch_bounds__(MAX_THREAD_COUNT) void gather_custom(
    float* input, uint32_t* index, float* gather_output, uint32_t in_width, uint32_t index_total_length)
{
    // Calculate global thread ID
    int32_t out_row = blockIdx.x * blockDim.x + threadIdx.x;

    // Maps to the row index of output tensor
    if (out_row >= index_total_length) {
        return;
    }
    // Single thread processes entire row (all columns) - enables coalesced memory access
    uint32_t in_row = index[out_row];
    int input_idx = in_row * in_width;
    int output_idx = out_row * in_width;
    for (int32_t col = 0; col < in_width; col++) {
        gather_output[output_idx] = input[input_idx];
        input_idx += 1;
        output_idx += 1;
    }
}
```

CANN

SIMT算子编程实操： 以gather算子为例

数据切分: 简单的均匀切分

```
uint32_t MAX_THREAD_COUNT = 2048; // 设置核函数最大线程数为2048
uint32_t AIV_COUNT = platformInfoMgr->GetCoreNumAiv(); // AIV物理核数
uint32_t num_blocks ; // 启动的线程块个数，根据输入shape动态分配，最大不超过65536
uint32_t thread_num_per_block = 0; // 每个线程块启动的线程数，根据输入shape动态分配，最大不超过2048
```

- 当input的行数M较小时，优先将线程块数达到物理核数

```
num_blocks = AIV_COUNT;
thread_num_per_block = M / AIV_COUNT;
```

- 当input的行数M较大时，优先将线程数打满到最大线程数

```
thread_num_per_block = MAX_THREAD_COUNT ;
num_blocks = M / thread_num_per_block;
```

CANN

SIMT算子编程实操：以gather算子为例

Host调用示例： 使用内核调用符<<<>>>

```
int32_t main(int32_t argc, char* argv[]) {
    // aclInit
    aclInit(nullptr);
    aclrtSetDevice(device_id);
    aclrtCreateStream(&stream);
    // Malloc memory in host and device
    …
    uint32_t dyn_ubuf_size = 0;  // No need to alloc dynamic memory.

    // Call kernel funtion with <<<...>>>
    gather_custom<<<num_blocks, thread_num_per_block, dyn_ubuf_size, stream>>>(
        input_device, index_device, output_device, in_shape[1], index_total_length);

    aclrtSynchronizeStream(stream);
    // Copy result from device to host
    …
    // Free memory
    …
    // aclDeInt
    aclrtDestroyStream(stream);
    aclrtResetDevice(device_id);
    aclFinalize();
}
```

CANN

本章小结

- **以gather算子为例，介绍基于SIMT开发算子的端到端流程**
  - ✓ gather算子功能介绍
  - ✓ 算子核函数实现
  - ✓ 以简单的均匀切分为例，介绍算子切分计算设计基本思路
  - ✓ 算子Host侧调用示例

CANN

目录

Part 1 下一代硬件架构回顾

Part 2 SIMT编程模型介绍

Part 3 SIMT算子编程实操

Part 4 SIMT算子优化技巧

SIMT算子优化技巧(1)：合理的切分，充分利用SIMT算力

- 运行时具体的线程数推荐使用32的整数倍，最大值为2048。如果超出寄存器限制会报错。
- 访存类算子线程数越多越好
- 计算类算子512~1024线程即可，优先保证不发生栈溢出
- 对于索引和偏移量能使用b32的就不要使用b64(通用寄存器是32bit的，使用b64类型会占用两个寄存器，寄存器的数量翻倍)

注意：寄存器溢出后续会提供编译选项，可以在编译日志中查看是否出现寄存器溢出

```
// 简单均匀切分方式，适合访存类算子
bool block_splite(uint32_t index_total_length, uint32_t &numBlocks, uint32_t &thread_num_per_block) {
    uint32_t real_core_num = 0;
    const auto& platformInfoMgr = platform_ascendc::PlatformAscendCManager::GetInstance();
    if (platformInfoMgr == nullptr) {
        return false;
    }
    real_core_num = platformInfoMgr->GetCoreNumAiv();
    numBlocks = real_core_num;
    thread_num_per_block= (index_total_length + numBlocks -1) / numBlocks;
    if (thread_num_per_block > MAX_THREAD_COUNT) {
        thread_num_per_block = MAX_THREAD_COUNT;
        numBlocks = (index_total_length + thread_num_per_block - 1) / thread_num_per_block;
        if (numBlocks > MAX_BLOCK_COUNT) {
        return false;
        }
    }
    return true;
}
```

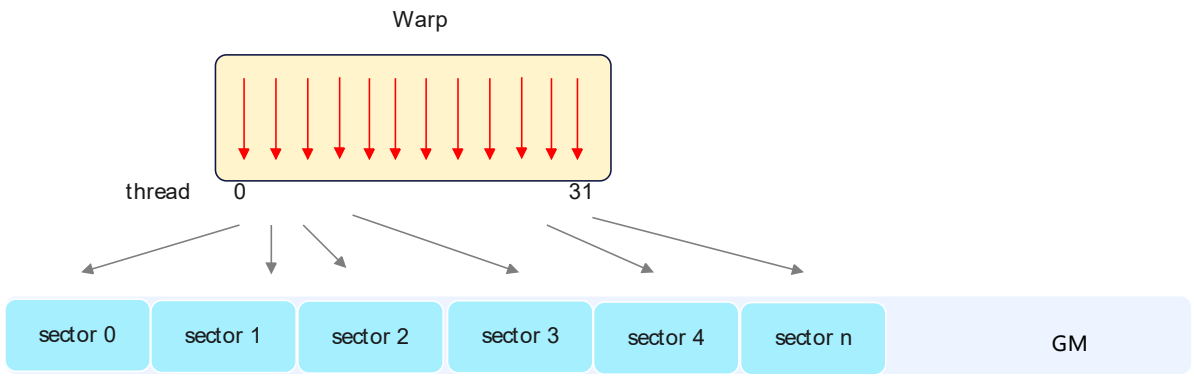
线程个数	寄存器个数
2048	16
1024	32
512	64

CANN

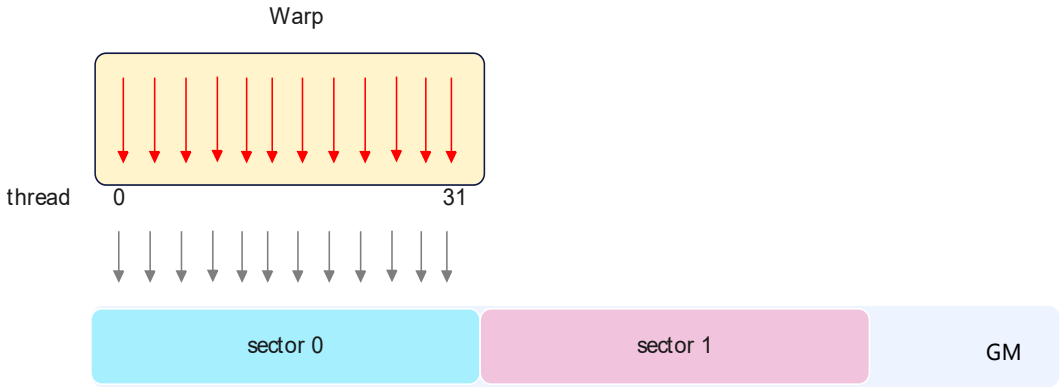
SIMT算子优化技巧(2)：访存优化，提升带宽效率

访存合并

同一个Warp内的线程对全局内存的访问请求，如果落在同一个sector范围内可以合并为同一个内存请求



warp内线程落在n个sector中



warp内线程落在同1个sector中

最优情况是warp内所有线程落在同一个sector中，此时带宽使用率最高

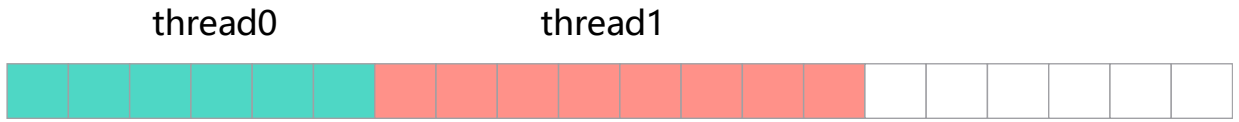
CANN

SIMT算子优化技巧(2)：访存优化，提升带宽效率

计算顺序优化

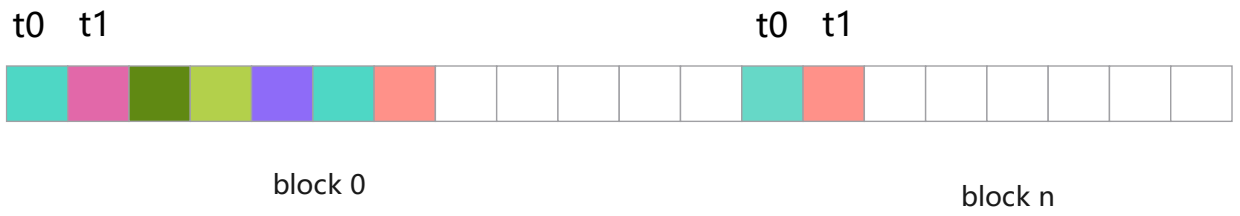
下面以一个向量加为例，演示运算顺序对性能的影响
计算过程为Z = X + Y，向量加的总数据量为32 * 8192
gridDim设为32，blockDim设置为1024，即每个线程计算8个元素

- 切分方式1：按照thread切分数据，每个thread计算连续的8个元素，每个warp的访存数据不连续




```
int32_t tid = blockIdx.x * blockDim.x + threadIdx.x;
for (int k = 0; k < POINTS_PER_THREAD; k++) {
    int32_t idx = tid * POINTS_PER_THREAD + k;
    if (idx < total_length) {
        z[idx] = x[idx] + y[idx];
    }
}
```

- 切分方式2：按照block切分数据，确保每个warp的访问数据连续




```
Int32_t start_id = blockIdx.x * POINTS_PER_BLOCK +
threadIdx.x
for (int k = 0; k < POINTS_PER_THREAD; k++) {
   int32_t idx = start_id + k * blockDim.x;
   if (idx < total_length) {
      z[idx] = x[idx] + y[idx];
   }
}
```

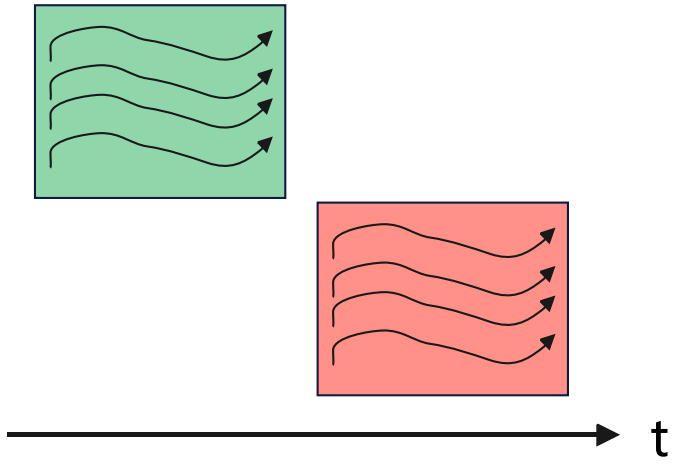
CANN

SIMT算子优化技巧(3)：避免在warp内进行分支判断

SIMT硬件单元为每个 Warp分配一个程序计数器(PC)，如果warp出现分支跳转，会导致分支串行执行

```
// 同一Warp内线程执行不同分支，不推荐
if (threadIdx.x % 2 == 0) {
    d_data[idx] *= 2; // 一半线程执行
} else {
    d_data[idx] += 1; // 另一半线程执行
}
```

```
int warp_id = threadIdx.x / 32;
if (warp_id % 2 == 0) {
    d_data[idx] *= 2; // 整Warp执行
} else {
    d_data[idx] += 1; // 整Warp执行
}
```



需要分支判断时，优先考虑Warp间的分支判断，保证同一Warp内无分支跳转，充分利用计算资源

CANN

SIMT算子优化技巧(4)：优化代码写法及编译提示，配合编译优化

对独立的运算操作按类别进行分组，将多个独立没有依赖的load/计算/store操作分组放到一起，有利于编译器优化代码，提高性能

```
    // LOAD(x) -> PROCESSING(x) -> STORE(x) and
    // LOAD(y) -> PROCESSING(y) -> STORE(y)
    int tid = threadIdx.x;
    for (int j=tid; j<SIZE; j+=num_threads) {
        float x = global_tensor_inout1(j);
        x = exp(x);
        ...
        global_tensor_inout1(j) = x + 1.0f;
        float y = global_tensor_inout2(j);
        y = ln(y);
        ...
        global_tensor_inout2(j) = y - 1.0f;
    }
} );
```

推荐写法

```
    // LOAD(x and y) -> PROCESSING(x and y) -> STORE(x and y)
    int tid = threadIdx.x;
    for (int j=tid; j<SIZE; j+=num_threads) {
        float x = global_tensor_inout1(j);
        float y = global_tensor_inout2(j);
        x = expf(x);
        ...
        y = lnf(y);
        ...
        global_tensor_inout1(j) = x + 1.0f;
        global_tensor_inout2(j) = y - 1.0f;
    }
} );
```

CANN

SIMT算子优化技巧(4)：优化代码写法及编译提示，配合编译优化

使用pragma unroll优化性能

循环次数为编译期常量时，可以开启pragma unroll，提示编译器进行循环展开

优点：

- for循环展开后，可以减少不必要的变量定义和分支判断
- 代码更线性，提高指令级并行

缺点：

可能会造成程序代码膨胀、以及增加寄存器消耗

```
#pragma unroll
            for (uint8_t i = 0; i < rank; i++) {
                T2 index = src1Tensor[indicesCurrentIdx * rank + i];
                if (index < 0) {
                    index += inputShape[i];
                }
                if (index >= inputShape[i] || index < 0) {
                    idxOutOfBound = true;
                    break;
                }
                srcIdx = srcIdx * inputShape[i] + index;
            }
            if (idxOutOfBound) {
                SetOutOfBoundValue<T1>(dstTensor, idx);
            } else {
                srcIdx = srcIdx * lastDimSize + gatherAxisIdx;
                dstTensor[idx] = src2Tensor[srcIdx];
            }
        } else {
#pragma unroll
            for (uint8_t i = 0; i < rank; i++) {
                T2 index = src1Tensor[indicesCurrentIdx * rank + i];
                if (index >= inputShape[i] || index < 0) {
                    idxOutOfBound = true;
                    break;
```

CANN

SIMT算子优化技巧(5)：优化代码写法及编译提示，配合编译优化

控制入参数量，避免性能劣化

SIMT的入参使用寄存器进行存储，其大小为28*32Bit；
对于超出大小限制的参数，会使用预留空间进行中转传递。超过这个大小会使用其他地址空间中转传递，导致访问延迟加大。

建议：
1. 参数数量较多时，如果能在Kernel内计算得到的参数，不建议作为参数传入；
2. 如果确实参数较多，可以申请静态内存空间，将这一部分参数搬入UB(如较大的tiling参数结构体，不建议在函数入口处展开)；

CANN

本章小结

- 介绍了一些Ascend C SIMT算子开发优化技巧
  - ✓ 合理的切分，充分利用SIMT算力
  - ✓ 访存优化，提升带宽效率
  - ✓ 避免在warp内进行分支判断
  - ✓ 优化代码写法及编译提示，配合编译优化

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann